Question 1

Part A

Proof. Let $n \in \mathbb{Z}$, and let `L` be an arbitrary list of length n. Let $k \in \mathbb{Z}$. We will prove that the function `encode` will make at least n comparisons with taking into consideration the associated helper function. To do this, we will first show that the best-case running time of `encode` is in $\mathcal{O}(n)$, and then show it is in $\Omega(n)$. Once we've shown that the best-case running time is in $\mathcal{O}(n)$ and in $\Omega(n)$, we will have shown that the best-case running time is tightly bound by n, i.e, in $\Theta(n)$.

First, we will show that the lower bound of the best-case running time for the function `encode` is exactly n where the running time is defined to be the number of comparisons.

First, note that there are two comparisons in the entirety of the given code, specifically, on lines 12 and 4. On line 12, there is one comparison that will be run n times due to being enclosed in a loop on line 11. There are no breaks or early return statements, and therefore, the loop will run to it's full length unavoidably. On line 4, there is another comparison that is inside a helper function which is only called in the case the comparison on line 12 evaluates to false. Therefore, in the best-case scenario, the helper function will not run at all and therefore avoid the additional comparisons. In such a case, since the loop on line 11 iterates exactly n times, the comparison on line 12 will be executed n times. This shows that the lower bound best-case running time is in $\Omega(n)$.

Next, we will show that the upper bound of the best-case running time for the function `encode` is exactly n where the running time is defined to be the number of item comparisons.

To do so, we consider l_n to be the set of lists for input of length n where each integer of the aforementioned lists are the same value as k. l_n will serve as the input family for `L`.

Since all inputs for of the input family l_n is equal to k, the else never evaluates which avoids having to run the comparison within the helper function `find_position`. Therefore, since the comparison on line 12 will occur n times given an input from the input family l_n, the best-case running time of `encode` is in $\mathcal{O}(n)$.

Because we've shown that the best-case running time is in $\mathcal{O}(n)$ and $\Omega(n)$, we know the best-case running time is in $\Theta(n)$.

Part B

Proof. Let $n \in \mathbb{Z}$, and let `L` be an arbitrary list of length n. Let $m \in \mathbb{Z}$ and let `M` be an arbitrary list of length m. Assume $m \geq n$. Let $k \in \mathbb{Z}$. We will prove that the function `encode` will make at most $n(m+1)$ comparisons, or in other words, have a worst-case running time in $\Theta(nm)$. To do this, we will prove the upper and lower bounds of the worst-case, i.e, $\mathcal{O}(nm)$, $\Omega(nm)$.

First, we will show the upper bound of worst-case of the function is $n(m+1)$. We will also show how $n(m+1)$ steps, defined as comparisons, translates to the functions running time being in $\mathcal{O}(n^2)$.

To begin, note that there exists two comparisons on lines 4 and 12. Then, we see that line 4 is nested in a loop that iterates m times at most per execution of `find_position`. `find_position` executes n times since it's called in the loop on line 11 since we are assuming the worst-case scenario where the comparison on line 12 evaluates to false. Additionally, the comparison on line 12 iterates n times due to the loop on line 11. Therefore, The maximum number of comparisons is $n + nm = n(m+1)$ times.

Next, we will show the lower bound of the worst-case of the function is $n(m+1)$. To do so, let q_n be the input family such that k does not exist in `L`, and no elements in `M` exists as elements in `L`.

In such a case, the comparison on line 12 will always evaluate to false since there does not exist an element in `L` that is equal to k. This will cause `find_position` to always execute in addition to the aforementioned comparison on line 12. `find_position` contains a loop that will iterate for m times since the comparison on line 4 always evaluates to false. This consistent false evaluation occurs since the two items being compared are the items over the list `M` and an item in `L` both passed in from `encode` on line 15. Our input family q_n specifies that there does not exist any elements that are both in `M` and `L` and therefore, no elements in one may be equal to an element in another. Since the loop on line 11 iterates n times, both the comparison on line 12 and `find_function` is executed n times (i.e, $n(1 + WC_{find_function})$). the loop in `find_position` runs m times, so the comparison on line 4 runs m times. In other words, $n(1+m)$.

Since we have shown the upper bound and lower bound of the worst-case runtime to be $n(1+m) = n+nm$, we can conclude that the worst-case runtime of `encode` to be in $\Theta(nm)$.

Part C

Proof. We will prove that the average running time of `encode` is $\Theta(nm)$ when all possible values are integers beween 0 to 9 inclusive and the probability of selecting any digit is evenly distributed, and therefore, 1 in 10. Let $n \in \mathbb{Z}$, and let `L` be an arbitrary list of length n. Let $m \in \mathbb{Z}$ and let `M` be an arbitrary list of length m. Assume $m \geq n$. Let $k \in \mathbb{Z}$. Let T be the random variable that represrents the running times of `encode`.

We will start by evaluating the expected runtime of `find_function`. This function returns once a specific item is found in `M`.Assuming that the probability of any digit from 0 to 10 have the same probability of occurring, we can assume there exists m cases such that there is a binomially distributed chance of each index up to and including m to be the first occurrence of a given target. The number of steps (comparisons) it takes to get to the first occurrence is equal to the index of the first occurrence.

$$\begin{aligned}
E[T_f] &= \sum_t t_m(x) P(T = t) \\
&= \sum_{i=1}^{m} i(\frac{9}{10})^{i-1}(\frac{1}{10}) \\
&= \sum_{i=1}^{m} i(\frac{9}{10})^{i}(\frac{1}{9}) \\
&= (\frac{1}{9}) \sum_{i=1}^{m} i(\frac{9}{10})^{i} \\
&= (\frac{1}{9})(\frac{(\frac{9}{10})^{m+1}(m(\frac{-1}{10}) - 1) + \frac{9}{10}}{(\frac{1}{10})^2}) \\
&= \frac{100}{9}((\frac{9}{10})^{m+1}(\frac{-11}{10}m) + \frac{9}{10})
\end{aligned}$$

With this, we can then calculate the expected running time of `encode`. Note that the comparison on line 12 always executes. The number of times `find_function` is executed depends on a $\frac{9}{10}$ probability that a given element in `L` is equal to k. From this, we can see there exists n cases, one case for each of the number of times `find_function` executes. There also exists the case where `find_function` never runs.

$$
\begin{aligned}
E[T] &= \sum_t t_{nm}(x)P(T=t) \\
&= n(\frac{1}{10})^n + \sum_{i=1}^{n} E[T_f](\frac{9}{10})^i(\frac{1}{10})^{n-i}\binom{n}{i} \\
&= n(\frac{1}{10})^n + \sum_{i=1}^{n}(\frac{100}{9}((\frac{9}{10})^{m+1}(\frac{-11}{10}m) + \frac{9}{10}))(\frac{9}{10})^i(\frac{1}{10})^{n-i}\binom{n}{i}
\end{aligned}
$$

From this, we can see that the average-case running time is $\Theta(nm)$.

Part D

Proof. Let $n \in \mathbb{Z}$, and let `L` be an arbitrary list of length n. Let $m \in \mathbb{Z}$ and let `M` be an arbitrary list of length m. Assume $m \geq n$. Let $k \in \mathbb{Z}$. Let T be the random variable that represrents the running times of `encode`. We will prove that the average running time of `encode` is $\Theta(nm)$ when all possible values for lists `L` and `M` are integers sampled without replacement from a set ranging from 1 to $n-1$ inclusive. k is sampled from a set of integers from 0 to n inclusive.

To start, we will find the expected running time of `find_function`. Note that there are n positions in which the function may return. Each iteration results in one comparison. Note again that we are sampling without replacement.

Then,

$$
\begin{aligned}
E[T_f] &= \sum_{i=1}^{m} i(\frac{{}_{n-1}P_i}{{}_nP_i}) \\
&= \sum_{i=1}^{m} i\frac{\frac{(n-1)!}{(n-1-i)!}}{\frac{n!}{(n-1)!}} \\
&= \sum_{i=1}^{m} i\frac{(n-1)!(n-i)!}{n!(n-1-i)!} \\
&= \sum_{i=1}^{m} i\frac{(n-i)!}{n(n-i-1)!} \\
&= \sum_{i=1}^{m} i\frac{(n-i)}{n} \\
&= m - \frac{m(m+1)}{2n}
\end{aligned}
$$

With this, we can calculate the expected running time of `encode`. Note that the comparison on line 12 always executes. There exists 2 cases. The first that there is a $\frac{1}{n+1}$ chance that `find_function` will not run. Otherwise, since all digits are unique, (drawn without replacement), there will exist exactly 1 value

k that will be equal to a value in `L`. This means that there is a $\frac{n}{n+1}$ probability that `find_function` in this case, must run $n-1$ times.

$$\begin{aligned} E[T] &= n(\frac{1}{n+1}) + (\frac{n}{n+1})(n-1)E[T_f] \\ &= n(\frac{1}{n+1}) + (\frac{n}{n+1})(n-1)(m - \frac{m(m+1)}{2n}) \end{aligned}$$

Therefore, the average case runtime for `encode` is in $\Theta(nm)$.

Question 2

Part A

I believe the best abstract data type for this problem is a priority Queue. A priority queue defines insertion, finding the maximum value, and extracting the maximum value. In this hypothetical situation, two priority queues may be used. One containing all disribution centres that have a surplus of dosages, and one that has a shortage of dosages. Then, the central office can use the extract max operation of the priority queue ADT for both the surplus ADT and the shortage ADT. We are assuming that the shortage ADT does not use negative values. I.e, if there one distribution center has a shortage by 10% the value stored will be 10, not -10.